Politechnika Krakowska im. Tadeusza Kościuszki
Wydział Informatyki i Matematyki

Projekt 4

Zaawansowane Techniki Kryptografii i Kryptoanalizy

Identyfikacja aplikacji w szyfrowanym kanale HTTPS/TLS na podstawie rozmiaru pakietów (k-NN)

Adrian Sajdak
Numer albumu: 146992

Jan Wojnicki
Numer albumu: 147020

Spis treści

Streszczenie 4
1. Wprowadzenie 4
1.1. Kontekst i motywacja 4
1.2. Cel badawczy i pytania badawcze 4
1.3. Status realizacji projektu 5
2. Podstawy teoretyczne 5
2.1. Co ujawnia szyfrowany kanał HTTPS/TLS i QUIC 5
2.2. Analiza ruchu i fingerprinting 5
2.3. Algorytm k-NN 5
2.4. Padding jako mechanizm obronny 6
3. Formalny model zagrożeń 6
3.1. Profil przeciwnika 6
3.2. Interfejs dostępny dla przeciwnika 6
3.3. Zasada Kerckhoffsa 6
3.4. Ilorazowe kryterium sukcesu 6
3.5. Założenia i ograniczenia modelu 7
4. Organizacja pracy zespołu 7
4.1. Role w zespole 7
4.2. Podział zadań i postęp prac 7
4.3. Repozytorium i zasady pracy z Git 8
4.4. Macierz wkładu (Contribution Matrix) 8
5. Dane 9
5.1. Przegląd dostępnych źródeł danych 9
5.2. Zbiór UC Davis QUIC 9
5.3. Pobieranie i kontrola jakości 9
5.4. Mapowanie aplikacji na kategorie ruchu 10
5.5. Analiza eksploracyjna i wykryte artefakty 10
6. Metodyka badawcza 12
6.1. Architektura potoku przetwarzania 12
6.2. Ekstrakcja cech 12
6.3. Podział danych i zapobieganie wyciekom 13
6.4. Grupa kontrolna 13
6.5. Dobór hiperparametrów 13
6.6. Metryki oceny 13
7. Wyniki eksperymentów 14
7.1. Wyniki strojenia k-NN 14
7.2. Ocena na zbiorze testowym i kryterium sukcesu 14
7.3. Porównanie z grupą kontrolną 15
7.4. Wpływ zestawu cech i liczby pakietów 15
7.5. Test na niezależnych partycjach zbioru 16

7.6. Porównanie z modelami referencyjnymi ... 17
8. Testy odporności na padding ... 18
8.1. Modele obrony ... 18
8.2. Scenariusze przeciwnika ... 19
8.3. Wyniki i koszt obrony ... 19
9. Analiza bezpieczeństwa i dyskusja ... 21
9.1. Interpretacja wyników w modelu zagrożeń ... 21
9.2. Przesunięcie danych i artefakty zbierania ruchu ... 22
9.3. Walidacja na ruchu TLS po TCP ... 23
9.4. Zagrożenia dla trafności badań ... 25
9.5. Zalecenia obronne ... 25
10. Podsumowanie i wnioski ... 26
Bibliografia ... 26
Załącznik A. Instrukcja uruchomienia ... 27
Załącznik B. Dziennik zmian ... 28

Streszczenie

Szyfrowanie TLS i QUIC chroni treść komunikacji, ale nie ukrywa rozmiarów, kierunków i czasów pakietów. Projekt sprawdza, czy te metadane wystarczają biernemu podsłuchującemu do rozpoznania rodzaju ruchu: wideo, audio, www lub transferu plików. Klasyfikatorem jest k-NN działający wyłącznie na pierwszych pakietach przepływu. Badania prowadzimy na rzeczywistym ruchu QUIC ze zbioru UC Davis i na ruchu TLS po TCP ze zbioru CESNET-TLS22.

Na chronologicznym zbiorze testowym UC Davis klasyfikator osiągnął Accuracy 99,46% (95% CI 98,9%–99,7%), a na drugim tygodniu CESNET-TLS22 92,52%, co spełnia kryterium 90% z instrukcji projektu. Analiza danych ujawniła jednak artefakty skryptów zbierających ruch i silne przesunięcie danych: na przepływach wygenerowanych przez ludzi trafność spada do 80,7%. Las losowy i SVM zachowują się podobnie.

Symulowany padding obniża trafność przeciwnika, który nie zna obrony, nawet do 37,4%. Przeciwnik znający obronę osiąga jednak przy każdym z dziewięciu wariantów co najmniej 96,90%, korzystając głównie z odstępów czasu. Sam padding rozmiarów nie chroni więc przed analizą ruchu.

Słowa kluczowe: analiza ruchu sieciowego, szyfrowanie, TLS, QUIC, k-NN, fingerprinting, kanał boczny, padding, przesunięcie danych.

1. Wprowadzenie

1.1. Kontekst i motywacja

Większość ruchu w sieciach firmowych jest dziś szyfrowana, przez co klasyczna inspekcja treści traci zastosowanie. Administrator nadal musi jednak planować pasmo i wykrywać usługi używane bez zgody działu IT (Shadow IT). Pozostaje mu analiza metadanych: rozmiarów pakietów, ich kierunków i odstępów czasu.

Ta sama technika jest kanałem bocznym naruszającym prywatność użytkowników, ponieważ ujawnia charakter aktywności mimo poprawnie działającej kryptografii [1], [4]. Projekt bada skalę tego wycieku i sprawdza, czy obrona polegająca na wypełnianiu pakietów (padding) go ogranicza.

1.2. Cel badawczy i pytania badawcze

Celem projektu jest zbudowanie i ocena klasyfikatora k-NN rozpoznającego kategorię ruchu (wideo, audio, www, pliki) wyłącznie na podstawie metadanych pakietów. Kryterium sukcesu z instrukcji projektu to Accuracy > 90% na zbiorze testowym. Przyjęliśmy następujące pytania badawcze:

- **PB1.** Czy k-NN osiąga Accuracy > 90%, obserwując tylko początek przepływu?
- **PB2.** Które metadane niosą najwięcej informacji i ile pakietów potrzeba do rozpoznania ruchu?
- **PB3.** W jakim stopniu wynik zależy od cech aplikacji, a w jakim od artefaktów sposobu zbierania danych?
- **PB4.** Czy padding obniża skuteczność ataku i jakim kosztem?

1.3. Status realizacji projektu

Tabela 1 zestawia etapy z instrukcji projektu z obecnym stanem prac. Wszystkie etapy i zadania z tabeli 4 zostały zrealizowane.

Tabela 1. Stan realizacji etapów projektu

Etap z instrukcji	Stan	Opis
1. Przechwycenie ruchu lub pobranie danych	Zrealizowany	Dane pobrane i zweryfikowane (rozdział 5), walidacja na CESNET-TLS22 (podrozdział 9.3)
2. Wyznaczenie wektorów cech	Zrealizowany	Cechy sekwencyjne i statystyczne (podrozdział 6.2)
3. Trening k-NN	Zrealizowany	Strojenie i wyniki (rozdziały 6 i 7)
4. Testy odporności na padding	Zrealizowany	Dwa scenariusze przeciwnika, dziewięć obron (rozdział 8)
Raport IEEE/ACM i obrona	Zrealizowany	Sprawozdanie, prezentacja i przewodnik po projekcie

2. Podstawy teoretyczne

2.1. Co ujawnia szyfrowany kanał HTTPS/TLS i QUIC

TLS 1.3 [7] szyfruje dane aplikacji i większość uzgadniania, jednak rozmiar rekordu pozostaje bliski rozmiarowi danych, a nagłówki IP i TCP są jawne. QUIC [8] przenosi te mechanizmy nad UDP i korzysta z uzgadniania TLS 1.3 [9]. Szyfruje też większość nagłówka transportowego, więc ukrywa między innymi numery pakietów, ale nie ukrywa liczby, rozmiaru, kierunku ani czasu datagramów.

Zbiór UC Davis pochodzi z 2018 roku, czyli sprzed standaryzacji QUIC w RFC 9000 (2021), dlatego zawiera ruch wcześniejszej odmiany protokołu stosowanej przez Google. W przyjętym modelu przeciwnika TLS po TCP i QUIC udostępniają te same rodzaje metadanych, jednak różnice nie są pomijalne, ponieważ TCP inaczej potwierdza, dzieli i retransmituje dane. Z tego powodu przeprowadziliśmy dodatkową walidację na ruchu TLS po TCP (podrozdział 9.3).

Pole SNI w wiadomości ClientHello ujawnia nazwę serwera, więc w praktyce byłoby najprostszym źródłem informacji. Rozszerzenie Encrypted Client Hello [10] ma je ukryć. Nasz model celowo pomija SNI, aby badać informację, która zostaje po jego wyeliminowaniu.

2.2. Analiza ruchu i fingerprinting

Fingerprinting ruchu polega na rozpoznawaniu usług po charakterystycznych wzorcach metadanych. Hayes i Danezis [1] pokazali skuteczność szerokiego zestawu cech statystycznych w metodzie k-fingerprinting, a Wang i in. [4] zastosowali k-NN z uczonymi wagami cech do rozpoznawania stron WWW. Nasze cechy statystyczne nawiązują do [1].

Rezaei i Liu [2] klasyfikowali ruch QUIC na podstawie próbek pakietów i zaznaczyli, że ruch generowany skryptami może mieć inny rozkład niż ruch ludzki. Finamore i in. [3] potwierdzili to w replikacji badań na tym samym zbiorze: na partycji pochodzącej od ludzi obserwowali spadek accuracy o około 20 punktów procentowych i przypisali go przesunięciu danych.

2.3. Algorytm k-NN

Algorytm k-NN [6] nie buduje jawnego modelu. Dla nowej próbki x wyszukuje k najbliższych przykładów uczących i wybiera klasę z największą sumą głosów: $\hat{y}(x) = \operatorname{argmax}_c \sum_{i \in N_k(x)} w_i \cdot 1[y_i = c]$. Waga głosu w_i jest stała albo równa odwrotności odległości. Badamy odległość euklidesową i manhattańską.

k-NN jest wrażliwy na skalę cech, dlatego standaryzujemy je wyłącznie na podstawie danych uczących. Metoda słabnie też w wysokich wymiarach, co ma znaczenie dla surowych sekwencji pakietów: przesunięcie sekwencji o jedną pozycję zmienia wiele współrzędnych naraz. Zaletą k-NN jest prostota i brak założeń o rozkładzie danych, a kosztem jest czas wyszukiwania rosnący z liczbą przykładów.

2.4. Padding jako mechanizm obronny

Padding dopisuje bajty wypełnienia, aby osłabić związek między rozmiarem pakietu a treścią. TLS 1.3 przewiduje wypełnianie rekordów [7], a QUIC udostępnia ramki PADDING [8]. Dyer i in. [5] pokazali jednak, że obrony tego typu nie ukrywają zgrubnych cech ruchu, takich jak łączna objętość w każdym kierunku. Rozdział 8 weryfikuje tę hipotezę dla naszego klasyfikatora.

3. Formalny model zagrożeń

3.1. Profil przeciwnika

Przeciwnikiem jest **bierny podsłuchujący** (Passive Network Eavesdropper) umieszczony na ścieżce ruchu, na przykład na bramie sieci firmowej lub porcie lustrzanym przełącznika. Obserwuje pakiety, ale ich nie modyfikuje, nie opóźnia i nie wstrzykuje. Nie zna kluczy sesji i nie ma dostępu do urządzeń końcowych. Jego celem jest przypisanie przepływu do jednej z kategorii ze zbioru C = {wideo, audio, www, pliki}.

3.2. Interfejs dostępny dla przeciwnika

Poziom dostępu odpowiada atakowi z samym szyfrogramem (COA) ograniczonemu do metadanych. Przepływ f modelujemy jako ciąg krotek (t_i, l_i, d_i), gdzie t_i to czas, l_i to rozmiar, a $d_i \in \{+1, -1\}$ to kierunek i-tego pakietu. Widokiem przeciwnika jest prefiks $V_N(f)$ złożony z N pierwszych krotek, a atakiem jest funkcja A: $V_N(f) \to C$.

Tabela 2. Informacje dostępne przeciwnikowi

Element	Dostępny	Uzasadnienie
Rozmiary, kierunki i czasy pakietów	Tak	Wynikają z obserwacji nagłówków IP/TCP/UDP
Treść aplikacji	Nie	Chroniona szyfrowaniem TLS/QUIC
Klucze sesji	Nie	Przeciwnik jest bierny i nie kontroluje urządzeń końcowych
SNI, DNS, adresy IP serwerów	Pominięte celowo	Badamy informację pozostającą po ich ukryciu, np. przez ECH [10]

3.3. Zasada Kerckhoffsa

Zakładamy pełną jawność protokołów, cech, klasyfikatora oraz kodu źródłowego. Poufność treści zależy wyłącznie od kluczy sesji i atak jej nie narusza. Wyciek dotyczy metadanych, których szyfrowanie z założenia nie ukrywa. W testach odporności przyjmiemy to samo założenie wobec obrony: przeciwnik zna mechanizm paddingu i może uczyć klasyfikator na ruchu z wypełnieniem.

3.4. Ilorazowe kryterium sukcesu

Podstawowym wskaźnikiem jest Accuracy = |poprawne klasyfikacje| / |zbiór testowy|. Atak uznajemy za udany, gdy **Accuracy > 90%** na zbiorze testowym. Dodatkowo podajemy 95-procentowy przedział ufności, makro-F1 oraz przewagę nad klasyfikatorem trywialnym, liczoną jako $\mathrm{Adv} = (\mathrm{Acc} - \mathrm{Acc}_0) / (1 - \mathrm{Acc}_0)$, gdzie Acc_0 to trafność klasy większościowej.

3.5. Założenia i ograniczenia modelu

Model ma świadomie przyjęte ograniczenia:

- Rozważamy zamknięty zbiór kategorii (closed-world), bez ruchu nieznanych usług.
- Każdy przepływ jest obserwowany osobno, bez łączenia wielu połączeń jednego użytkownika.
- Analiza odbywa się po przechwyceniu ruchu, bez ograniczeń czasu rzeczywistego.
- Wszystkie usługi w zbiorze należą do Google, więc wynik nie obejmuje różnic między dostawcami.

4. Organizacja pracy zespołu

4.1. Role w zespole

Tabela 3. Role w zespole

Osoba	Rola	Główny obszar odpowiedzialności
Adrian Sajdak	Kierownik projektu, analityk kryptograficzny	Model zagrożeń, architektura kodu, cechy, potok k-NN, koordynacja prac, prezentacja
Jan Wojnicki	Inżynier danych, analityk bezpieczeństwa	Dane i ich jakość, analiza eksploracyjna, grupa kontrolna, padding i testy odporności, walidacja na TLS, analiza przesunięcia danych, modele referencyjne, raport końcowy

4.2. Podział zadań i postęp prac

Wagi w tabeli 4 odzwierciedlają szacowany nakład pracy i sumują się do 100. Wszystkie zadania zostały zrealizowane.

Tabela 4. Podział zadań i postęp prac

ID	Zadanie	Odpowiedzialny	Waga	Status
Z1	Analiza wymagań, plan i podział pracy	A. Sajdak	3	Zrealizowane
Z2	Podstawy teoretyczne i model zagrożeń	A. Sajdak	8	Zrealizowane
Z3	Przegląd i wybór źródeł danych	J. Wojnicki	4	Zrealizowane
Z4	Struktura repozytorium, konfiguracja, środowisko	A. Sajdak	3	Zrealizowane
Z5	Pobieranie danych, parser, kontrola jakości	J. Wojnicki	7	Zrealizowane
Z6	Analiza eksploracyjna i wykrycie artefaktów	J. Wojnicki	6	Zrealizowane
Z7	Ekstrakcja cech sekwencyjnych i statystycznych	A. Sajdak	8	Zrealizowane
Z8	Podział danych, potok k-NN i strojenie	A. Sajdak	9	Zrealizowane
Z9	Grupa kontrolna	J. Wojnicki	4	Zrealizowane
Z10	Struktura sprawozdania i opis metodyki	Obie osoby	4	Zrealizowane
Z11	Implementacja mechanizmów paddingu	J. Wojnicki	8	Zrealizowane
Z12	Testy odporności k-NN na padding	J. Wojnicki	10	Zrealizowane
Z13	Walidacja na ruchu TLS po TCP (CESNET-TLS22)	J. Wojnicki	6	Zrealizowane
Z14	Pogłębiona analiza przesunięcia danych	J. Wojnicki	4	Zrealizowane
Z15	Porównanie z modelami referencyjnymi i istotność cech	J. Wojnicki	3	Zrealizowane
Z16	Analiza bezpieczeństwa i wnioski	J. Wojnicki	5	Zrealizowane
Z17	Raport końcowy IEEE/ACM i macierz wkładu	J. Wojnicki	5	Zrealizowane
Z18	Prezentacja i przygotowanie do obrony	Obie osoby	3	Zrealizowane

Na etap końcowy (od 11.11.2026) zmieniliśmy podział pracy, aby wyrównać nakład obu osób. Zadania Z12, Z14, Z16 i Z17 przejął J. Wojnicki, a A. Sajdak koordynuje prace i przygotowuje prezentację. Zadanie Z18 pozostaje wspólne.

4.3. Repozytorium i zasady pracy z Git

Kod i historia zmian znajdują się w repozytorium https://github.com/AdrianSajdak/KryptografiaProjekt4. Praca odbywa się na gałęzi `main`, w małych commitach opisanych jednym zdaniem. Pliki danych nie są przechowywane w repozytorium, bo odtwarzają je skrypty. Zależności są przypięte w pliku `requirements.txt`, a poprawność modułów weryfikują testy `pytest`.

4.4. Macierz wkładu (Contribution Matrix)

Macierz wkładu zestawia podział zadań z tabeli 4 z historią repozytorium (stan na commit `203df4e`, ostatni przed dodaniem macierzy). Wagi zadań wspólnych dzielimy po równo. Tabela 5 podaje udział każdej osoby, a tabela 6 liczbę commitów w obszarach projektu.

Tabela 5. Macierz wkładu członków zespołu

Osoba	Zadania	Udział wg wag	Commity	Linie dodane / usunięte
Adrian Sajdak	Z1, Z2, Z4, Z7, Z8, wspólnie Z10 i Z18	34,5%	47 (58%)	6 234 / 0
Jan Wojnicki	Z3, Z5, Z6, Z9, Z11, Z12, Z13, Z14, Z15, Z16, Z17, wspólnie Z10 i Z18	65,5%	34 (42%)	7 748 / 29

Tabela 6. Liczba commitów w obszarach projektu

Obszar	Adrian Sajdak	Jan Wojnicki
Dane i ich jakość	9	2
Cechy	3	0
Modele, eksperymenty, analizy	6	9
Testy	4	5
Wyniki i wykresy	8	8
Konfiguracja i środowisko	5	3
Dokumentacja	13	8

Pracujemy wspólnie przy jednym stanowisku. W pierwszym etapie wszystkie commity, także dotyczące zadań Z3, Z5, Z6 i Z9 przypisanych J. Wojnickiemu, powstawały z konta A. Sajdaka (8–9 listopada 2026 r.). W etapie końcowym commit podpisuje osoba odpowiedzialna za dane zadanie. Linie dodane obejmują wyłącznie pliki tekstowe, bez dokumentów i wykresów.

5. Dane

5.1. Przegląd dostępnych źródeł danych

Przed wyborem danych porównaliśmy dostępne publicznie zbiory (tabela 7). Pierwszeństwo miał zbiór wskazany w instrukcji, ponieważ jego usługi można przypisać do czterech wymaganych kategorii ruchu.

Tabela 7. Porównanie kandydujących źródeł danych

Zbiór	Protokół	Zawartość	Dostępność	Decyzja
UC Davis QUIC [2]	QUIC	6 672 przepływy, 5 usług Google, pakiety z czasem, rozmiarem i kierunkiem	Google Drive autorów, bez rejestracji	Wybrany
CESNET-TLS22 [11]	TLS po TCP	141,7 mln przepływów, 191 usług, sekwencje początkowych pakietów	Zenodo, CC BY 4.0, 9,5 GB	Walidacja (podrozdział 9.3)
ISCX VPN-nonVPN	Mieszany	Ruch VPN i zwykły z 2016 r., pliki PCAP, również ruch nieszyfrowany	Strona UNB	Odrzucony: nie odzwierciedla współczesnego TLS
CSTNET-TLS1.3	TLS 1.3	120 klas witryn WWW	Publiczny	Odrzucony: etykiety witryn, nie rodzaje ruchu

5.2. Zbiór UC Davis QUIC

Zbiór przygotowali Rezaei i Liu [2] w laboratorium UC Davis. Ruch pięciu usług Google wygenerowano skryptami Selenium WebDriver i AutoIt, a niewielką część zebrano podczas rzeczywistej pracy ludzi. Każdy plik opisuje jeden przepływ i zawiera dla każdego pakietu znacznik czasu, czas względny, rozmiar i kierunek. Liczności przedstawia tabela 8.

Tabela 8. Liczba przepływów w partycjach zbioru

Aplikacja	Kategoria	pretraining (skrypt)	retraining (skrypt)	retraining (człowiek)
Google Music	audio	592	30	15
Google Drive	pliki	1 634	30	18
Youtube	wideo	1 077	30	20
Google Doc	www	1 221	30	15
Google Search	www	1 915	30	15
Razem		**6 439**	**150**	**83**

5.3. Pobieranie i kontrola jakości

Link z instrukcji (github.com/ucdavis/quic-data) zwraca błąd 404, dlatego dane pobieramy z folderu Google Drive wskazanego w repozytorium autorów zbioru. Skrypt sprawdza sumy SHA-256 archiwów, a parser czyta przepływy bezpośrednio z plików ZIP. Wyniki kontroli jakości zawiera tabela 9.

Tabela 9. Wyniki kontroli jakości danych

Kontrola	Wynik
Liczba przepływów i pakietów	6 672 przepływów, 44 544 105 pakietów
Zgodność liczności z dokumentacją zbioru	Zgodne we wszystkich partycjach i klasach
Duplikaty przepływów	0
Najkrótszy przepływ	176 pakietów
Zakres rozmiarów pakietów	54–1486 B
Pierwszy pakiet wysyła klient	100% przepływów
Pakiety poza kolejnością czasową	1 156 par w 938 przepływach, naprawione przez sortowanie

Ponieważ każdy przepływ zaczyna klient, wartość 1 w kolumnie kierunku interpretujemy jako ruch od klienta do serwera. Żaden przepływ nie powtarza się między partycjami, więc partycje retraining stanowią niezależne zbiory testowe.

5.4. Mapowanie aplikacji na kategorie ruchu

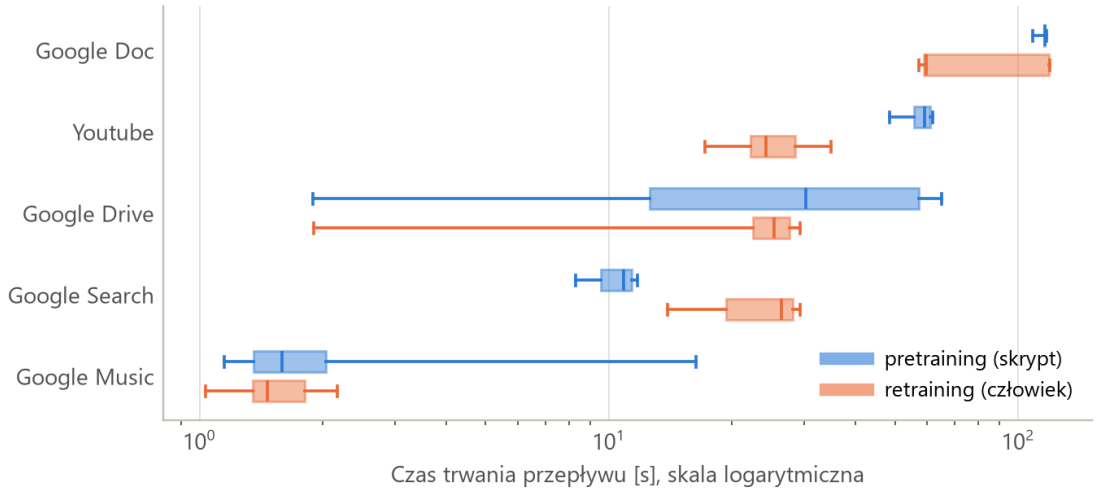
Instrukcja wymaga rozróżnienia czterech kategorii, a zbiór zawiera pięć usług. Przyjęte mapowanie przedstawia tabela 10. Google Doc traktujemy jako www, ponieważ jest interaktywną aplikacją webową opartą na krótkich żądaniach. Eksperymenty prowadzimy na obu poziomach etykiet.

Tabela 10. Mapowanie usług na kategorie ruchu

Usługa	Kategoria	Uzasadnienie
YouTube	wideo	Strumieniowanie wideo
Google Music	audio	Strumieniowanie muzyki
Google Search	www	Krótkie żądania i odpowiedzi stron WWW
Google Doc	www	Interaktywna aplikacja webowa
Google Drive	pliki	Transfer plików

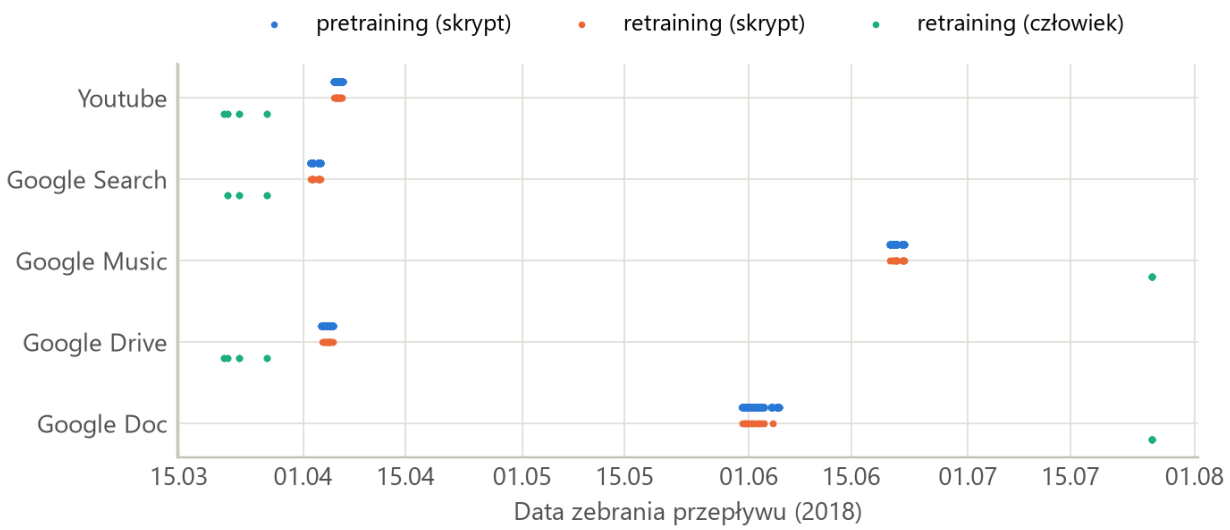
5.5. Analiza eksploracyjna i wykryte artefakty

Analiza eksploracyjna ujawniła dwie właściwości, które zadecydowały o projekcie eksperymentu. Po pierwsze, czas trwania przepływu jest niemal stały w obrębie klasy dla ruchu skryptowego: mediana wynosi 116,2 s dla Google Doc, 59,1 s dla YouTube i 10,9 s dla Google Search (rysunek 1). Odzwierciedla to scenariusz skryptu, a nie cechę aplikacji. W ruchu ludzkim te mediany wynoszą odpowiednio 59,5 s, 24,1 s i 26,4 s.



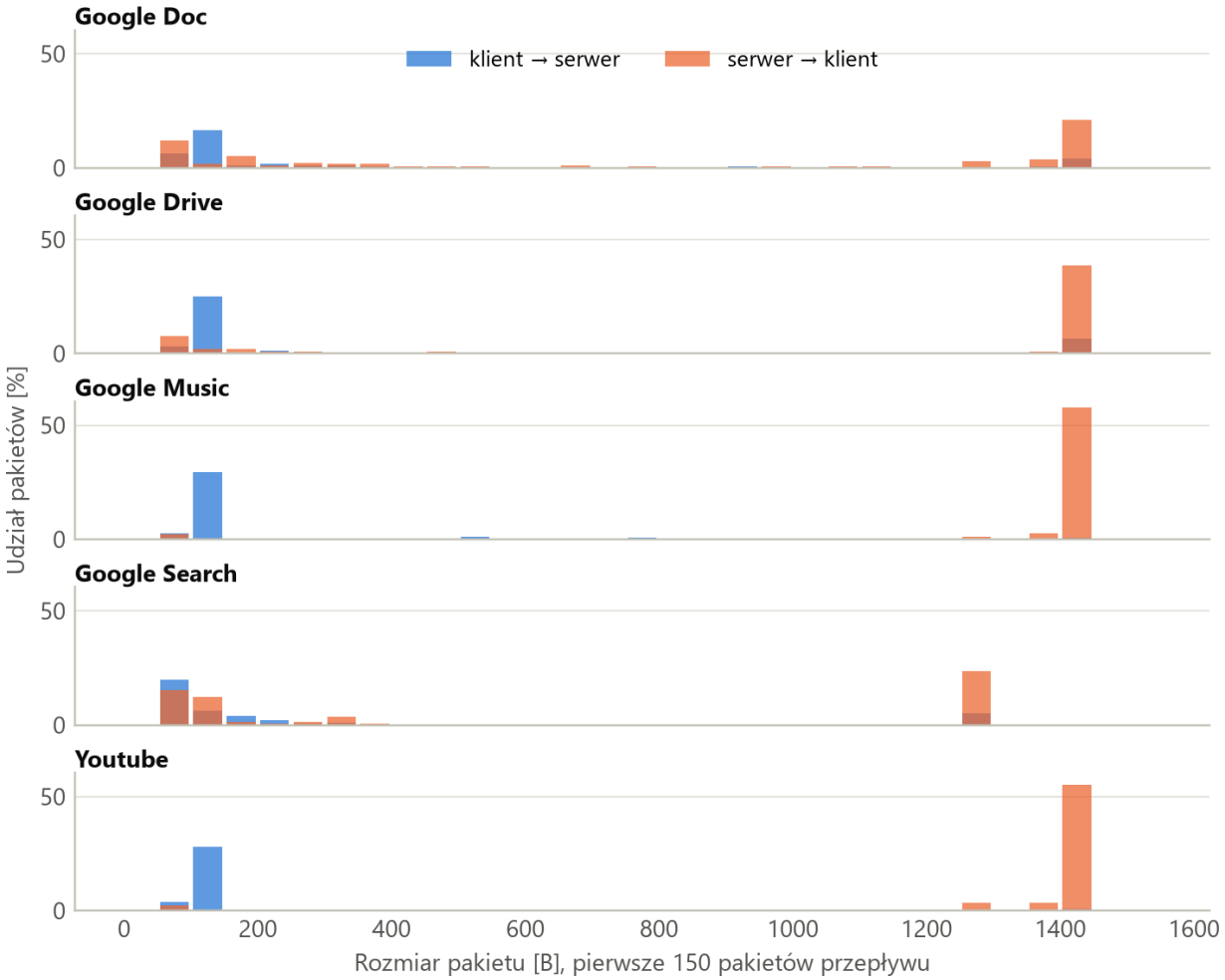
Rysunek 1. Czas trwania przepływów w ruchu skryptowym i ludzkim

Po drugie, każda klasa powstała w osobnym, nieprzecinającym się okresie (rysunek 2). Google Search zebrano 2–3 kwietnia 2018 r., Google Doc od 31 maja do 5 czerwca, a Google Music 20–22 czerwca. Klasa jest więc powiązana z okresem pomiaru, a tym samym z ówczesnymi warunkami sieci i wersją usługi. Wyniki na tym zbiorze mogą przez to zawyżać zdolność rozpoznawania aplikacji w innych warunkach.



Rysunek 2. Daty zebrania przepływów w partycjach zbioru

Rysunek 3 pokazuje różnice w rozkładach rozmiarów. Wideo, audio i pliki mają wyraźną przewagę dużych pakietów od serwera, a Google Search i Google Doc zawierają więcej małych pakietów w obu kierunkach. Najkrótszy przepływ ma 176 pakietów, więc prefiks do 150 pakietów nie wymaga dopełniania. Czas potrzebny na jego obserwację przedstawia tabela 11.



Rysunek 3. Rozkład rozmiarów pierwszych 150 pakietów w partycji pretraining

Tabela 11. Mediana czasu obserwacji pierwszych N pakietów [s], partycja pretraining

N	Google Doc	Google Drive	Google Music	Google Search	Youtube
10	0,29	0,41	0,06	0,60	0,04
25	0,32	2,55	0,07	0,75	0,19
50	0,36	6,14	0,11	0,84	0,20
100	1,68	7,31	0,12	0,91	0,29
150	3,79	7,91	0,16	1,17	0,33

6. Metodyka badawcza

6.1. Architektura potoku przetwarzania

Potok składa się z pięciu skryptów uruchamianych kolejno (tabela 12). Parametry eksperymentów znajdują się w pliku `configs/default.yaml`.

Tabela 12. Etapy potoku przetwarzania

Skrypt	Zadanie	Wynik
`01_download_data.py`	Pobranie zbioru	Archiwa ZIP po kontroli SHA-256
`02_build_dataset.py`	Parsowanie i kontrola jakości	Magazyn przepływów, raport jakości
`03_explore_data.py`	Analiza eksploracyjna	Tabele i wykresy z rozdziału 5
`04_run_knn.py`	Strojenie i ocena k-NN	Wyniki walidacji i testów
`05_plot_results.py`	Wizualizacja	Wykresy z rozdziału 7
`06_run_padding.py`	Testy odporności na padding	Wyniki z rozdziału 8
`07_analyze_shift.py`	Analiza przesunięcia danych	Wyniki z podrozdziału 9.2
`08_run_baselines.py`	Modele referencyjne	Wyniki z podrozdziału 7.6
`09_download_cesnet.py`	Pobranie CESNET-TLS22	Archiwum ZIP po kontroli MD5
`10_build_cesnet.py`	Losowanie próbki CESNET-TLS22	Magazyn 22 400 przepływów
`11_run_cesnet.py`	Walidacja na TLS po TCP	Wyniki z podrozdziału 9.3
`12_plot_final.py`	Wizualizacja etapu końcowego	Wykresy z rozdziałów 7–9

6.2. Ekstrakcja cech

Cechy liczymy wyłącznie z pierwszych N pakietów, $N \in \{10, 25, 50, 100, 150\}$. Nie używamy łącznej liczby pakietów, bajtów ani czasu trwania całego przepływu, ponieważ analiza z podrozdziału 5.5 wykazała, że odzwierciedlają one scenariusz skryptu. Rozmiar ze znakiem kierunku to $s_i = d_i \cdot l_i$, a odstępy czasu zapisujemy jako $\log(1 + \Delta t_i)$, gdzie Δt_i podano w milisekundach. Zestawy cech opisuje tabela 13.

Tabela 13. Badane zestawy cech

Zestaw	Zawartość	Wymiar
rozmiary	Sekwencja rozmiarów ze znakiem kierunku	N
odstępy czasu	Sekwencja zlogarytmowanych odstępów	$N - 1$
rozmiary + odstępy	Połączenie obu sekwencji	$2N - 1$
statystyki	Udziały kierunków, statystyki rozmiarów i odstępów (ogółem i w każdym kierunku), serie pakietów, histogramy rozmiarów, położenie pakietów wychodzących	94
wszystkie	Sekwencje i statystyki	$2N + 93$

6.3. Podział danych i zapobieganie wyciekom

Zbiór pretraining dzielimy chronologicznie w obrębie każdej aplikacji: najpóźniejsze 20% przepływów tworzy zbiór testowy (1 290 przepływów), a wcześniejsze 80% zbiór uczący (5 149). Sąsiednie przepływy powstawały w odstępach kilkudziesięciu sekund, więc podział losowy umieszczałby podobne obserwacje po obu stronach podziału. Podział chronologiczny odpowiada też sytuacji przeciwnika, który uczy się na wcześniejszym ruchu.

Walidację krzyżową prowadzimy w pięciu foldach ciągłych w czasie w obrębie aplikacji. Standaryzacja jest elementem potoku, więc skaler uczy się tylko na części uczącej każdego foldu. Zbiór testowy służy wyłącznie do końcowej oceny konfiguracji wybranych w walidacji. Dla porównania z literaturą podajemy też wynik podziału losowego, a partycje retraining wykorzystujemy jako niezależne zbiory testowe.

6.4. Grupa kontrolna

Grupa kontrolna (tabela 14) pozwala ocenić, czy wynik jest rzeczywisty i skąd pochodzi.

Tabela 14. Skład grupy kontrolnej

Klasyfikator	Co sprawdza
Klasa większościowa	Próg trywialny wynikający z niezrównoważenia klas
Losowanie według rozkładu klas	Poziom przypadku
k-NN na przetasowanych etykietach	Wyciek informacji w potoku: bez związku cech z klasą trafność powinna pozostać na poziomie trywialnym
k-NN na objętości całego przepływu	Skalę artefaktu: liczbę pakietów, bajtów i czas trwania przepływu jako jedyne cechy

6.5. Dobór hiperparametrów

Przeszukujemy pełną siatkę parametrów z tabeli 15, co daje 800 konfiguracji, a przy pięciu foldach 4000 treningów na każdy poziom etykiet. Konfigurację wybieramy według średniego makro-F1 w walidacji krzyżowej, ponieważ jest ono odporne na niezrównoważenie klas. Przy remisie preferujemy mniejszą liczbę pakietów.

Tabela 15. Siatka przeszukiwania

Parametr	Wartości
Zestaw cech	rozmiary, odstępy czasu, rozmiary + odstępy, statystyki, wszystkie
Liczba pakietów N	10, 25, 50, 100, 150
Liczba sąsiadów k	1, 3, 5, 7, 9, 11, 15, 21
Metryka odległości	euklidesowa, manhattańska
Wagi głosów	równe, odwrotność odległości

6.6. Metryki oceny

- **Accuracy** z 95-procentowym przedziałem ufności Wilsona jest wskaźnikiem kryterium sukcesu.
- **Makro-F1** i **balanced accuracy** równoważą wpływ klas o różnej liczności.
- **Precyzja, czułość i F1 dla każdej klasy** oraz **macierz pomyłek** pokazują strukturę błędów.

7. Wyniki eksperymentów

7.1. Wyniki strojenia k-NN

Najlepszą konfiguracją na obu poziomach etykiet okazał się zestaw **statystyki** dla N = 150, k = 3, z metryką manhattańska i wagami zależnymi od odległości. Dla czterech kategorii średnie makro-F1 w walidacji krzyżowej wyniosło 99,25%. Tabela 16 przedstawia najlepszą konfigurację każdego zestawu cech.

Tabela 16. Najlepsze konfiguracje zestawów cech (4 kategorie)

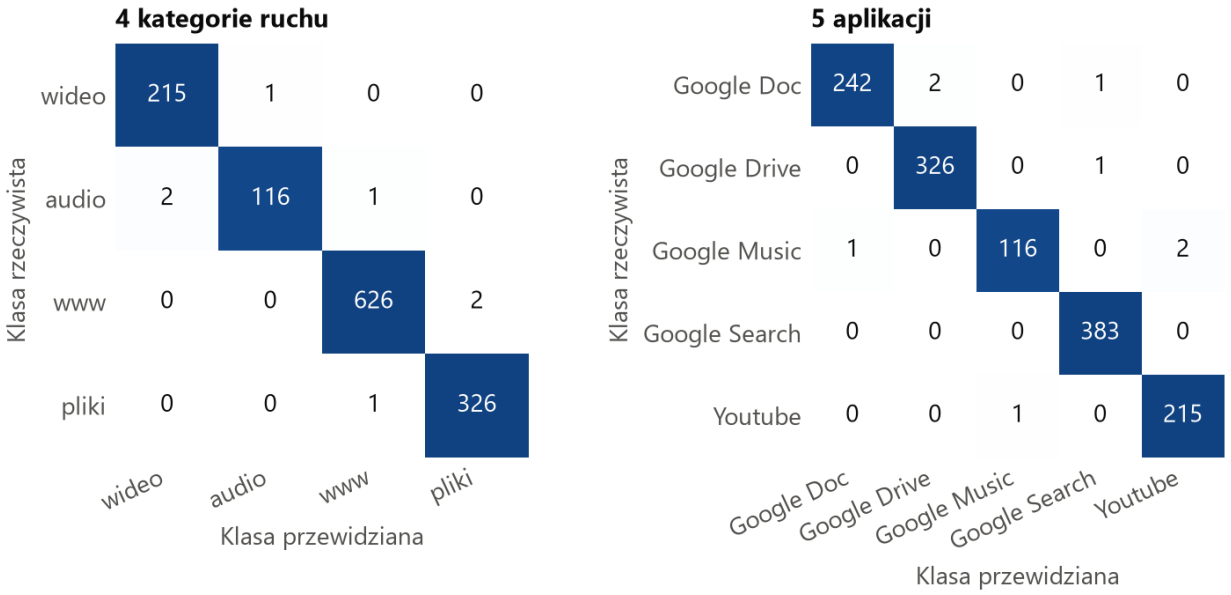
Zestaw cech	N	k	Metryka	Wagi	Makro-F1 CV	Accuracy test
statystyki	150	3	manhattańska	odległość	99,25%	99,46%
wszystkie	100	3	manhattańska	odległość	97,88%	98,14%
rozmiary + odstępy	10	1	manhattańska	równe	96,54%	98,14%
rozmiary	10	1	manhattańska	równe	95,93%	97,67%
odstępy czasu	10	1	manhattańska	równe	95,83%	97,05%

7.2. Ocena na zbiorze testowym i kryterium sukcesu

Na zbiorze testowym klasyfikator osiągnął Accuracy **99,46%** dla czterech kategorii. Dolna granica 95-procentowego przedziału ufności wynosi 98,9%, więc **kryterium Accuracy > 90% jest spełnione** dla tego zbioru. Przewaga nad klasą większościową to Adv = 98,9%. Wyniki zbiorcze zawiera tabela 17, a strukturę błędów rysunek 4.

Tabela 17. Wyniki na chronologicznym zbiorze testowym

Poziom etykiet	Próbek	Accuracy	95% CI	Balanced acc.	Makro-F1
4 kategorie ruchu	1290	99,46%	98,9%–99,7%	99,10%	99,21%
5 aplikacji	1290	99,38%	98,8%–99,7%	99,10%	99,21%



Rysunek 4. Macierze pomyłek na chronologicznym zbiorze testowym

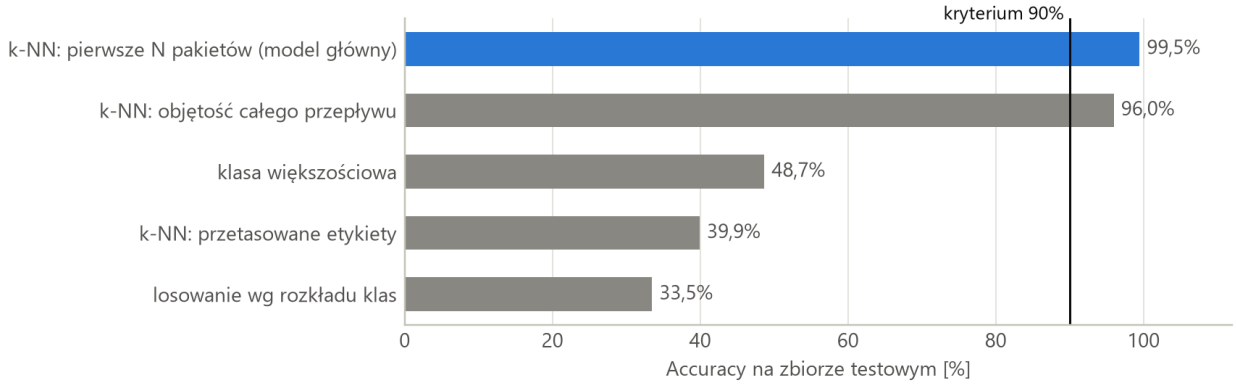
Tabela 18. Metryki dla kategorii ruchu na chronologicznym zbiorze testowym

Kategoria	Precyzja	Czułość	F1	Liczba próbek
wideo	99,1%	99,5%	99,3%	216
audio	99,1%	97,5%	98,3%	119
www	99,7%	99,7%	99,7%	628
pliki	99,4%	99,7%	99,5%	327

7.3. Porównanie z grupą kontrolną

Porównanie z grupą kontrolną przedstawia rysunek 5. k-NN z przetasowanymi etykietami osiąga 39,9%, czyli poniżej klasy większościowej (48,7%), co nie wskazuje na wyciek informacji w potoku.

Najważniejsza jest kontrola objętości: trzy liczby opisujące cały przepływ dają 96,0% trafności. Potwierdza to, że sama objętość i czas trwania wystarczają do osiągnięcia bardzo wysokiego wyniku na danych skryptowych. Dlatego model główny korzysta tylko z początku przepływu.

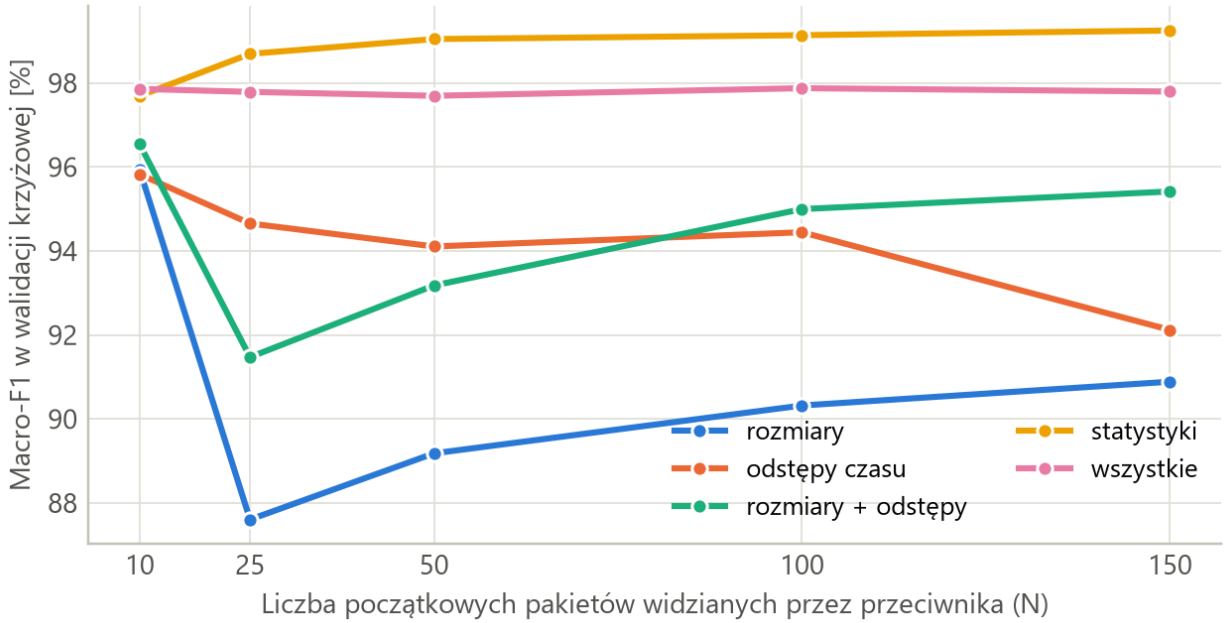


Rysunek 5. Accuracy modelu głównego i grupy kontrolnej (4 kategorie)

7.4. Wpływ zestawu cech i liczby pakietów

Rysunek 6 pokazuje wpływ liczby pakietów. Statystyki dają najlepsze wyniki dla każdego N z wyjątkiem N = 10 i poprawiają się wraz z N, osiągając 99,25%. Surowe sekwencje wypadają słabiej, co jest zgodne ze słabością k-NN w wysokich wymiarach i wrażliwością na przesunięcia sekwencji.

Sekwencje są wyraźnie lepsze dla N = 10 niż dla N = 25. Pierwsze pakiety obejmują uzgadnianie połączenia, którego przebieg zależy od serwera, więc mogą identyfikować serwer, a nie rodzaj ruchu. Jest to hipoteza do weryfikacji. Najlepsze N leży na granicy siatki, a jej rozszerzenie ogranicza długość najkrótszych przepływów.



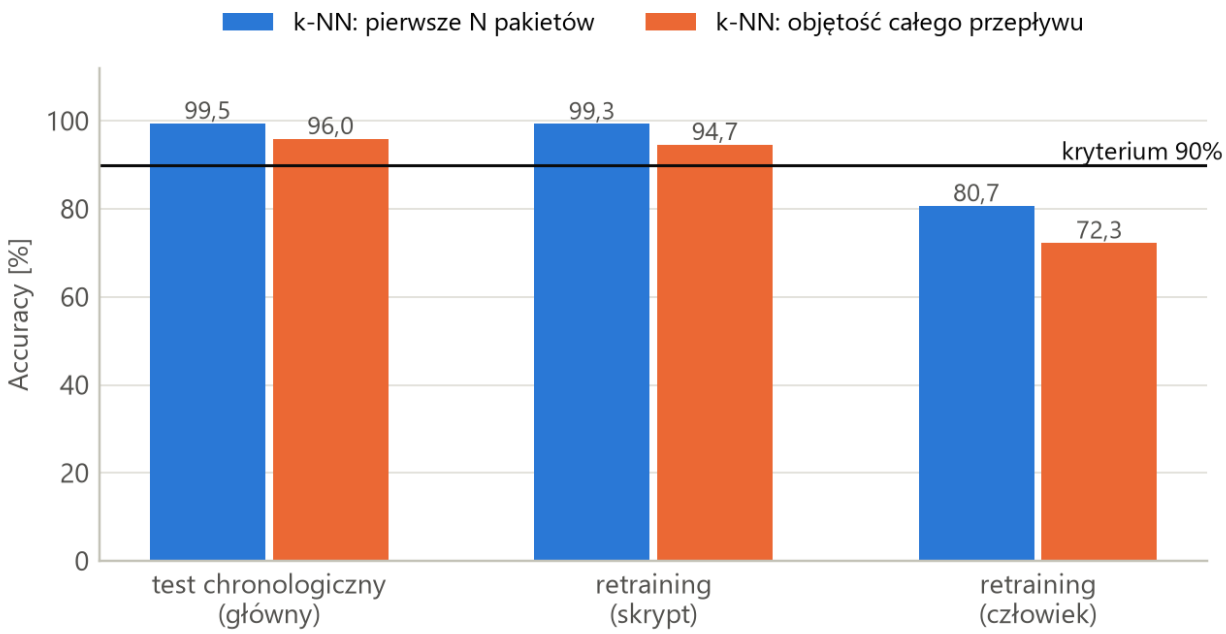
Rysunek 6. Makro-F1 w walidacji krzyżowej w zależności od zestawu cech i liczby pakietów (4 kategorie)

7.5. Test na niezależnych partycjach zbioru

Tabela 19 i rysunek 7 zestawiają zbiory testowe. Na niezależnej partycji skryptowej trafność pozostaje wysoka (99,33%). Na przepływach ludzkich spada do **80,7%** (95% CI 71,0%–87,8%), czyli **poniżej progu 90%**. Spadek zgodny jest ze zjawiskiem opisanym w [2], [3]. Kontrola objętości wypada na tej partycji gorzej (72,3%), ale przy 83 próbkach przedziały ufności są szerokie.

Tabela 19. Trafność modelu głównego na różnych zbiorach testowych (4 kategorie)

Zbiór testowy	Próbek	Accuracy	95% CI	Makro-F1	Accuracy kontroli objętości
Chronologiczny (główny)	1290	99,46%	98,9%–99,7%	99,21%	96,0%
Losowy	1288	99,53%	99,0%–99,8%	99,46%	nie badano
Retraining (skrypt)	150	99,33%	96,3%–99,9%	99,17%	94,7%
Retraining (człowiek)	83	80,72%	71,0%–87,8%	81,25%	72,3%



Rysunek 7. Accuracy modelu głównego i kontroli objętości na zbiorach testowych (4 kategorie)

Najwięcej błędów dotyczy kategorii pliki (czułość 61,1%) oraz www (czułość 76,7%). Na poziomie aplikacji Accuracy wynosi 77,1%, a najsłabiej rozpoznawane jest Google Search (czułość 33,3%). Autorzy zbioru wskazują, że ludzie wykonują działania nieobecne w skryptach, na przykład zmianę nazwy pliku [2]. Przyczyny spadku analizuje podrozdział 9.2.

7.6. Porównanie z modelami referencyjnymi

Aby sprawdzić, czy wyniki są specyficzne dla k-NN, porównaliśmy go z lasem losowym [17] i maszyną wektorów nośnych (SVM) z jądrem RBF [18]. Modele referencyjne dostają te same cechy (statystyki, N = 150), ten sam podział danych i te same foldy walidacji krzyżowej. Las losowy (300 drzew) stroimy na 4 konfiguracjach głębokości i liczby próbek w liściu, a SVM na 6 konfiguracjach parametrów C i γ. Wyniki zawiera tabela 20.

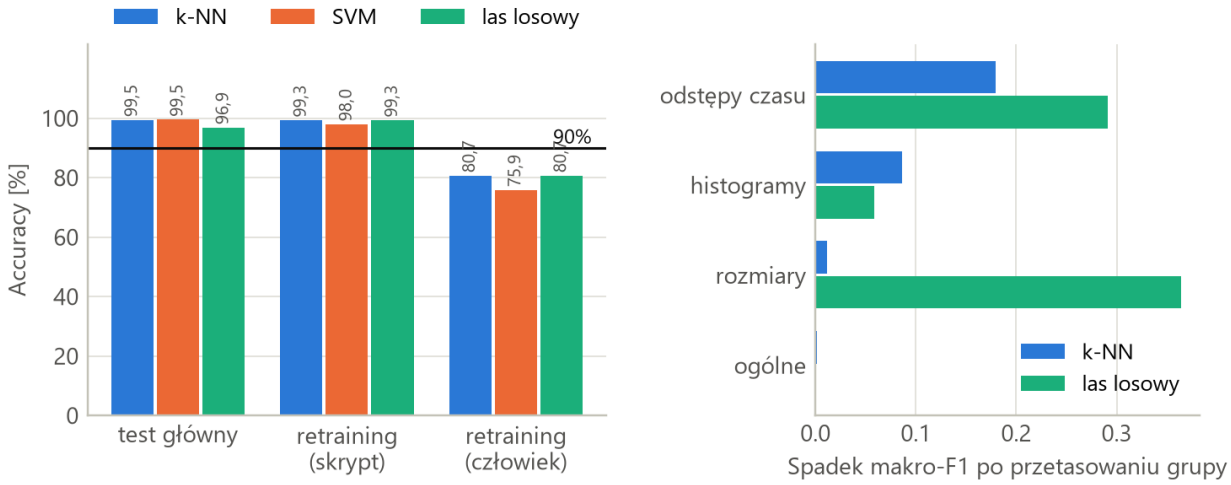
Tabela 20. Porównanie k-NN z modelami referencyjnymi (4 kategorie, statystyki, N = 150)

Model	Wybrane parametry	Makro-F1 CV	Test główny	Retraining (skrypt)	Retraining (człowiek)
k-NN	k = 3, manhattańska, wagi odl.	99,25%	99,46%	99,33%	80,72%
Las losowy	głębokość bez limitu, liść ≥ 1	98,33%	96,90%	99,33%	80,72%
SVM (RBF)	C = 10, γ = scale	99,46%	99,53%	98,00%	75,90%

SVM wypada minimalnie lepiej od k-NN na teście głównym (99,53%), ale najsłabiej na ruchu ludzi (75,90%). Las losowy jest najsłabszy na teście głównym (96,90%). Żaden model nie przekracza 90% na ruchu ludzi, więc spadek trafności nie wynika z wyboru klasyfikatora, lecz z różnicy między danymi uczącymi a testowymi (podrozdział 9.2).

Istotność cech mierzymy permutacyjnie na ostatnim foldzie walidacyjnym, więc zbiory testowe nie biorą w tym udziału. Wszystkie cechy jednej grupy tasujemy jednocześnie, ponieważ cechy są silnie skorelowane: przetasowanie pojedynczej cechy obniża makro-F1 k-NN najwyżej o 0,0028. Po przetasowaniu grup spadek makro-F1 k-NN wynosi 0,180 dla odstępów czasu, 0,087 dla histogramów, 0,011 dla statystyk rozmiarów i 0,002 dla cech ogólnych (rysunek 8). Las losowy korzysta głównie z rozmiarów (0,364) i odstępów czasu (0,291). Silna zależność k-NN od odstępów czasu tłumaczy zarówno

odporność przeciwnika adaptacyjnego na padding (rozdział 8), jak i wrażliwość na przesunięcie cech czasowych w ruchu ludzi (podrozdział 9.2).



Rysunek 8. Accuracy modeli na zbiorach testowych i istotność grup cech

8. Testy odporności na padding

8.1. Modele obrony

Obrony symulujemy przez przekształcenie rozmiarów pakietów w istniejącym zbiorze (moduł `defenses/padding.py`). Wypełnienie tylko zwiększa pakiet i nie może przekroczyć największego rozmiaru występującego w zbiorze, 1486 B, który pełni rolę MTU. Badamy trzy rodziny obron:

- **Stały rozmiar:** każdy pakiet dopełniamy do 1486 B. To najsilniejszy wariant wypełniania rekordów TLS 1.3 [7] i ramek PADDING w QUIC [8].
- **Wielokrotność bloku:** rozmiar zaokrąglamy w górę do wielokrotności $b \in \{64, 128, 256, 512\}$ B, co ukrywa dokładną długość, ale zostawia zgrubną.
- **Losowe wypełnienie:** do każdego pakietu dopisujemy losową liczbę bajtów z przedziału $[0, r]$, $r \in \{50, 100, 200, 400\}$ B, przy stałym ziarnie losowości.

Koszt obrony mierzymy narzutem przepustowości, czyli względnym przyrostem liczby bajtów wszystkich przepływów: $\eta = \Sigma\, l'_i \,/\, \Sigma\, l_i - 1$, gdzie l_i i l'_i to rozmiary pakietu przed obroną i po niej. Wartości narzutu zawiera tabela 21.

Tabela 21. Symulowane obrony i ich narzut przepustowości

Obrona	Parametr	Narzut przepustowości
Stały rozmiar	1486 B	36,6%
Wielokrotność bloku	b = 64 B	4,5%
Wielokrotność bloku	b = 128 B	5,8%
Wielokrotność bloku	b = 256 B	8,9%
Wielokrotność bloku	b = 512 B	14,2%
Losowe wypełnienie	r = 50 B	2,3%
Losowe wypełnienie	r = 100 B	4,4%
Losowe wypełnienie	r = 200 B	6,6%
Losowe wypełnienie	r = 400 B	9,5%

Symulacja ma ograniczenie: nie zmienia czasów ani kierunków pakietów. Prawdziwa obrona mogłaby nieco zmienić odstępy między pakietami, a obrony typu stały strumień danych, takie jak BuFLO [5],

maskują także czasy. Wyniki dla cech czasowych odpowiadają więc obronie, która nie zmienia rytmu wysyłania pakietów.

8.2. Scenariusze przeciwnika

Każdą obronę oceniamy w dwóch scenariuszach, które różnią się wiedzą przeciwnika (moduł `robustness.py`, skrypt `06_run_padding.py`):

- **Przeciwnik naiwny** nie wie o obronie. Używa modeli nauczonych na ruchu bez wypełnienia: modelu głównego oraz najlepszej konfiguracji każdego zestawu cech z tabeli 16. Klasyfikuje ruch po nałożeniu obrony.
- **Przeciwnik adaptacyjny** zna obronę, zgodnie z zasadą Kerckhoffsa (podrozdział 3.3). Nakłada ją na własne dane uczące i powtarza całą procedurę strojenia: 800 konfiguracji w pięciokrotnej blokowej walidacji krzyżowej, wybór według makro-F1.

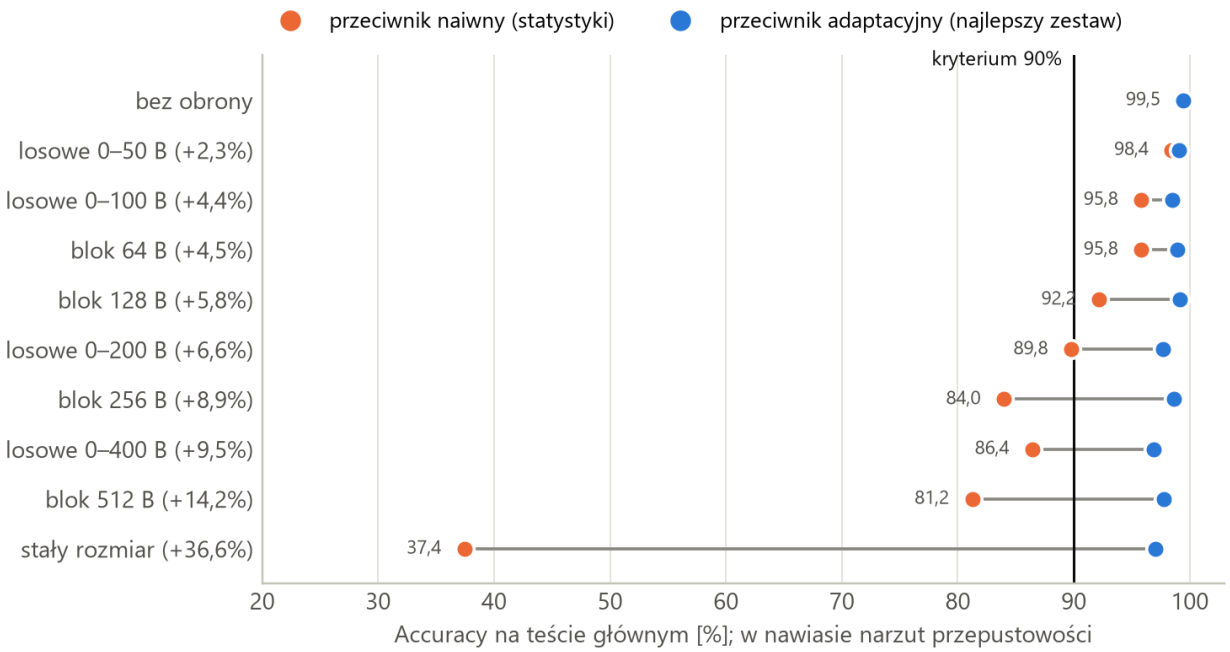
Obie wersje oceniamy na tych samych zbiorach testowych co model główny, po nałożeniu na nie obrony. Partycje retraining nie biorą udziału w strojeniu. Pełne obliczenia dla dziewięciu obron trwały 29 minut.

8.3. Wyniki i koszt obrony

Tabela 22 i rysunek 9 zestawiają trafność obu przeciwników na teście głównym. Dla przeciwnika naiwnego podajemy model główny (statystyki), a dla adaptacyjnego zestaw cech wybrany w jego walidacji krzyżowej.

Tabela 22. Trafność przeciwników przy poszczególnych obronach (4 kategorie)

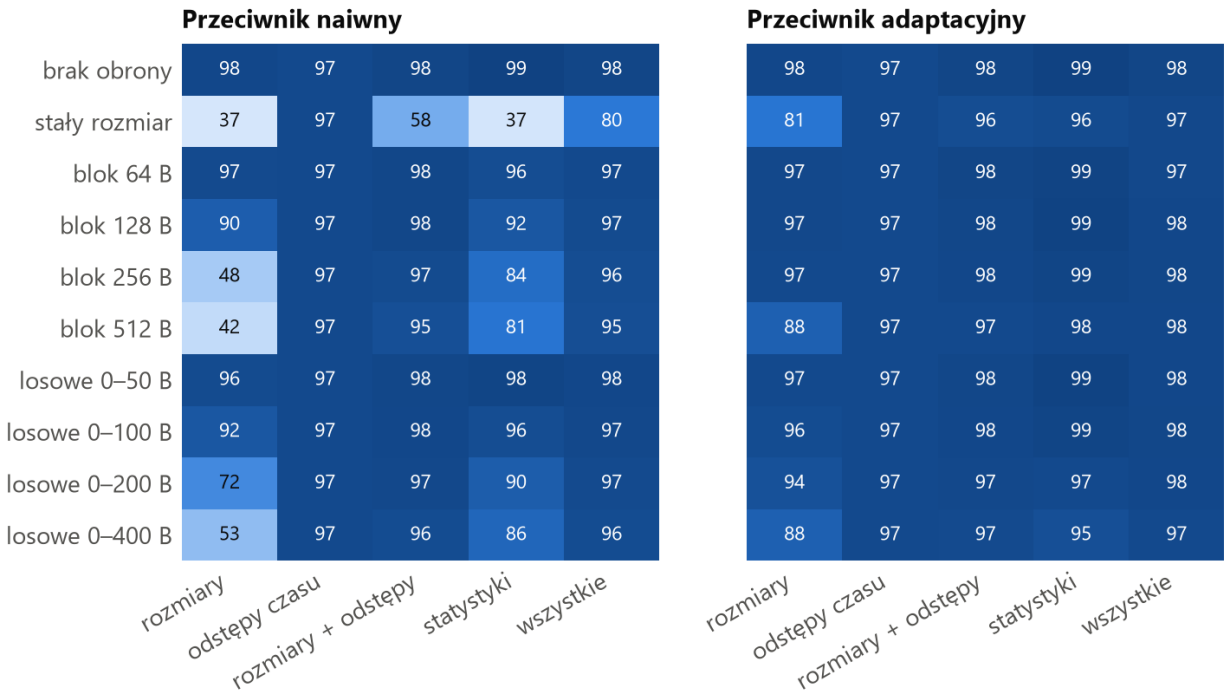
Obrona	Narzut	Naiwny: test główny	Adaptacyjny: zestaw cech	Adaptacyjny: test główny	Adaptacyjny: ruch ludzi
Brak obrony	0,0%	99,46%	statystyki	99,46%	80,72%
stały rozmiar	36,6%	37,44%	odstępy czasu	97,05%	68,67%
blok 64 B	4,5%	95,81%	statystyki	98,91%	80,72%
blok 128 B	5,8%	92,17%	statystyki	99,15%	83,13%
blok 256 B	8,9%	83,95%	statystyki	98,60%	79,52%
blok 512 B	14,2%	81,24%	statystyki	97,75%	77,11%
losowe 0–50 B	2,3%	98,45%	statystyki	99,07%	81,93%
losowe 0–100 B	4,4%	95,81%	statystyki	98,53%	79,52%
losowe 0–200 B	6,6%	89,77%	wszystkie	97,67%	80,72%
losowe 0–400 B	9,5%	86,43%	wszystkie	96,90%	81,93%



Rysunek 9. Trafność przeciwnika naiwnego i adaptacyjnego przy kolejnych obronach, uporządkowanych według narzutu

Przeciwnik naiwny traci tym więcej, im większy jest narzut obrony. Obrony o narzucie do 5% obniżają jego trafność najwyżej do 95,8%, blok 512 B do 81,2%, a stały rozmiar aż do 37,4%, czyli poniżej poziomu klasy większościowej. Odstępy czasu są na tę obronę obojętne: model z samymi odstępami zachowuje 97,1%.

Przeciwnik adaptacyjny odzyskuje niemal całą skuteczność. Najsłabszy wynik, 96,90%, uzyskuje przy obronie „losowe 0–400 B", więc **kryterium Accuracy > 90% jest spełnione przy każdej badanej obronie**. Przy stałym rozmiarze przeciwnik wybiera same odstępy czasu ($N = 10$, $k = 1$) i osiąga 97,05%. Nawet same kierunki pakietów, które zostają w zestawie rozmiarów po dopełnieniu do stałej długości, dają 80,9%. Na ruchu ludzi przeciwnik adaptacyjny osiąga 68,7% przy stałym rozmiarze, a przy pozostałych obronach od 77,1% do 83,1%, wobec 80,7% bez obrony.



Rysunek 10. Accuracy na teście głównym dla każdego zestawu cech, obrony i przeciwnika [%]

Rysunek 10 pokazuje wszystkie zestawy cech. Sekwencje rozmiarów są najbardziej podatne na padding, statystyki pośrednio, a odstępy czasu wcale. Wynik potwierdza hipotezę z pracy [5]: wypełnianie pakietów ukrywa dokładne rozmiary, ale nie usuwa informacji niesionej przez czasy, kierunki i zgrubną objętość. Najdroższa obrona, z narzutem 36,6%, chroni wyłącznie przed przeciwnikiem, który o niej nie wie.

9. Analiza bezpieczeństwa i dyskusja

9.1. Interpretacja wyników w modelu zagrożeń

W przyjętym modelu zagrożeń bierny podsłuchujący, znający tylko rozmiary, kierunki i czasy pakietów, rozpoznaje kategorię ruchu z trafnością 99,46% na zbiorze UC Davis (przewaga nad klasą większościową Adv = 98,9%) i 92,52% na ruchu TLS po TCP z CESNET-TLS22 (Adv = 90,0%). **Atak spełnia kryterium sukcesu w obu środowiskach.** Poufność treści pozostaje nienaruszona: przeciwnik nie odczytuje danych ani kluczy, ale poznaje charakter aktywności użytkownika.

Z punktu widzenia prywatności oznacza to, że szyfrowanie TLS i QUIC nie ukrywa, czy użytkownik ogląda wideo, słucha muzyki, przegląda strony czy przesyła pliki. Ukrycie nazwy serwera przez Encrypted Client Hello [10] tego nie zmienia, ponieważ nasz model w ogóle z niej nie korzysta. Z punktu widzenia administratora sieci ta sama technika umożliwia monitorowanie zużycia pasma i wykrywanie Shadow IT bez odszyfrowywania ruchu.

Zgodnie z zasadą Kerckhoffsa przeciwnik, który zna obronę, dostosowuje się do niej. Padding obniża trafność przeciwnika naiwnego nawet do 37,4%, ale przeciwnik adaptacyjny przy każdej badanej obronie osiąga co najmniej 96,90%. Bezpieczeństwo nie może więc opierać się na tym, że przeciwnik nie zna mechanizmu wypełniania.

Skuteczność ataku zależy jednak od tego, czy dane uczące przypominają atakowany ruch. Na przepływach ludzi trafność spada do 80,7%, a przyczyną jest przesunięcie danych opisane w podrozdziale 9.2. Praktyczny przeciwnik potrzebuje więc danych uczących z sieci i populacji użytkowników zbliżonych do celu ataku oraz okresowego ponownego uczenia.

9.2. Przesunięcie danych i artefakty zbierania ruchu

Partycję ludzką wykorzystujemy tu wyłącznie opisowo. Żaden wynik tej analizy nie wpływa na konfigurację modelu, więc partycja pozostaje niezależnym zbiorem testowym. Pomiary dotyczą przestrzeni, w której działa model główny: statystyk dla N = 150 po standaryzacji.

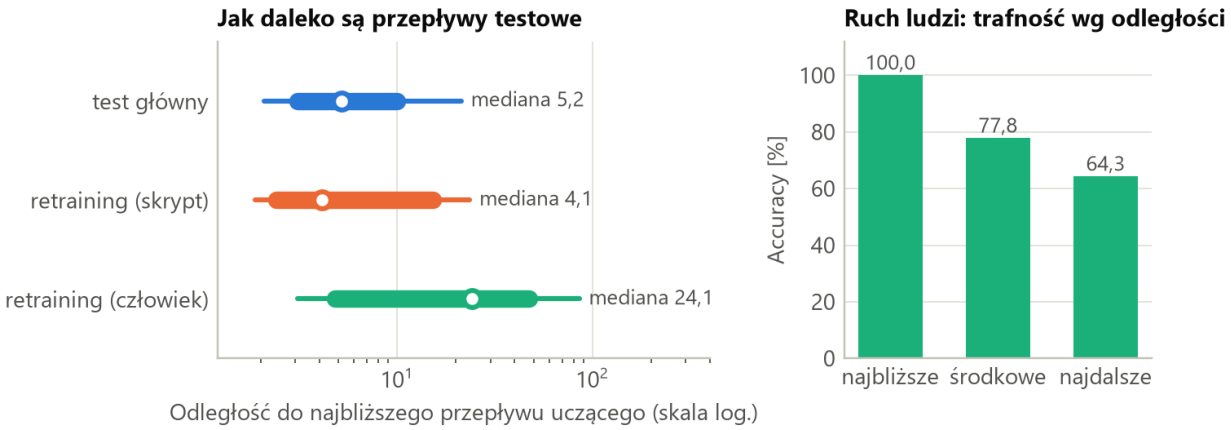
Przesunięcie rozkładów cech. Dla każdej cechy liczymy statystykę Kołmogorowa–Smirnowa [15] między zbiorem uczącym a zbiorem porównywanym, osobno w każdej kategorii, i uśredniamy wynik. Porównanie w obrębie kategorii oddziela zmianę rozkładu cech od zmiany proporcji klas. Statystyka przyjmuje wartość 0 dla identycznych rozkładów i 1 dla rozłącznych. Średnio wynosi 0,204 dla testu głównego i 0,271 dla ruchu ludzi (tabela 23).

Tabela 23. Średnia statystyka Kołmogorowa–Smirnowa względem zbioru uczącego w grupach cech

Grupa cech	Liczba cech	Test główny	Ruch ludzi
odstępy czasu	27	0,270	0,363
ogólne	10	0,229	0,319
rozmiary	27	0,181	0,258
histogramy	30	0,157	0,184

Najsilniej przesunięte są najkrótsze odstępy czasu. Dla kwantyla 10% odstępów wszystkich pakietów statystyka wynosi 0,59 w ruchu ludzi i 0,22 w teście głównym. Wskazuje to, że ruch ludzi zebrano w innych warunkach sieciowych lub na innym sprzęcie. Jest to istotne, bo k-NN opiera decyzje głównie na odstępach czasu (podrozdział 7.6).

Odległość od danych uczących. Mediana odległości do najbliższego przepływu uczącego wynosi 5,2 dla testu głównego, 4,1 dla partycji skryptowej i 24,1 dla ruchu ludzi (rysunek 11). Odległość przekłada się na błędy: po podziale ludzkich przepływów na tercyle odległości trafność wynosi kolejno 100,0%, 77,8% i 64,3%, po 27–28 przepływów w grupie.

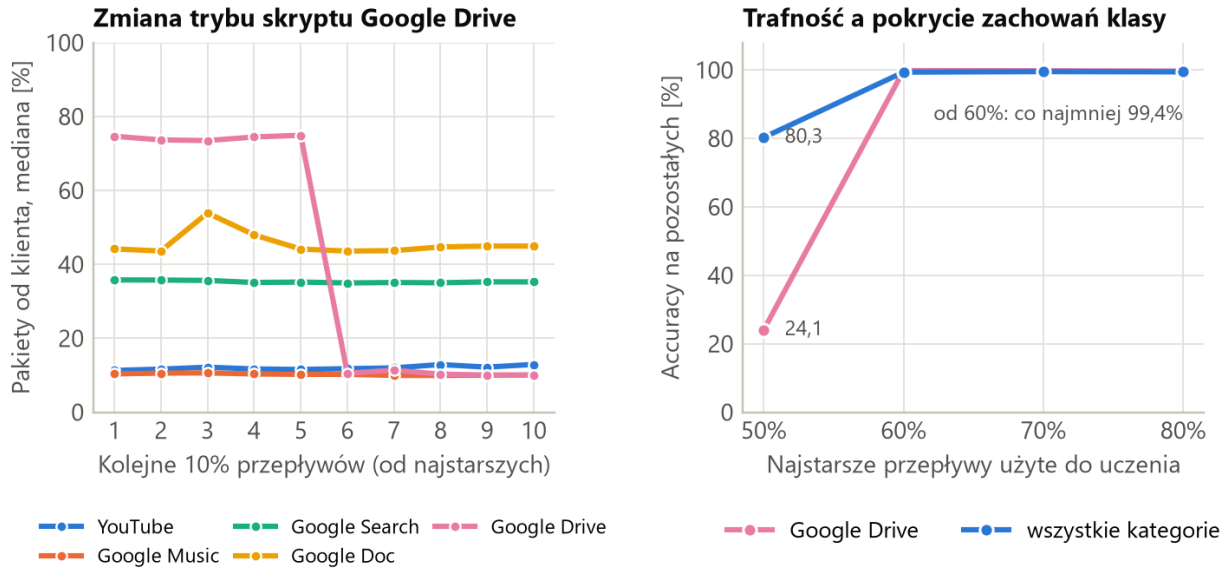


Rysunek 11. Odległość przepływów testowych od danych uczących i trafność na ruchu ludzi w tercylach odległości

Test klasyfikatora domeny. Zgodnie z ideą testów dwóch prób opartych na klasyfikatorze [16] uczymy k-NN odróżniać 83 przepływy ludzkie od tylu samo przepływów skryptowych tych samych aplikacji, w pięciokrotnej walidacji i 20 losowaniach. Średnia trafność wynosi 76,9% (odchylenie 2,9 p.p.). Ten sam test dla dwóch próbek skryptowych, zbioru testowego i uczącego, daje 64,8%. Ruch ludzi jest więc wyraźnie odróżnialny od skryptowego, a ślad zostawia nawet sam upływ czasu w danych skryptowych.

Dwa tryby skryptu Google Drive. Analiza pokrycia ujawniła kolejny artefakt. Gdy model uczy się na najstarszych 50% przepływów każdej aplikacji, trafność na pozostałych spada do 80,3%, a dla Google Drive do 24,1%. Przy 60% wraca do 99,4% (rysunek 12). Przyczyną jest zmiana zachowania skryptu w połowie pomiaru. W pierwszej połowie przepływów Google Drive 99,9% to wysyłanie plików (ponad

40% pakietów od klienta), a w drugiej przeważa pobieranie (wysyłanie stanowi tylko 8,8%). Pobieranie pliku przypomina w metadanych strumieniowanie wideo, dlatego model znający tylko wysyłanie myli je z YouTube.



Rysunek 12. Udział pakietów od klienta w kolejnych częściach danych i trafność w zależności od pokrycia zachowań klasy

Klasa pliki obejmuje więc dwa różne zachowania, a wysoki wynik wymaga, aby oba występowały w danych uczących. Podział 80/20 ten warunek spełnia: wysyłanie stanowi 66,5% zbioru uczącego i 5,8% testowego. Wśród 18 ludzkich przepływów Google Drive wysyłanie ma 12.

Spadek trafności na ruchu ludzi wynika zatem z przesunięcia danych, a nie z błędu potoku. Model działa dobrze w pobliżu danych uczących i słabo tam, gdzie brakuje mu podobnych przykładów. Pułapki z różnymi okresami pomiaru klas nie da się w tym zbiorze rozstrzygnąć, bo żadna klasa nie ma danych z okresu innej klasy.

9.3. Walidacja na ruchu TLS po TCP

Zbiór UC Davis zawiera ruch QUIC, dlatego metodę sprawdziliśmy też na ruchu TLS po TCP ze zbioru CESNET-TLS22 [11], [19]. Zebrano go w szkieletowej sieci CESNET2 od 4 do 17 października 2021 r. Zawiera 141,7 mln przepływów 191 usług, oznaczonych na podstawie nazw serwerów. Dla każdego przepływu zbiór podaje do 30 początkowych pakietów z danymi (rozmiar, kierunek i odstęp czasu) oraz objętość całego przepływu. Archiwum (9,5 GB) pobieramy z Zenodo, sprawdzamy jego sumę MD5 i czytamy bezpośrednio z pliku ZIP (moduł `data/cesnet.py`, skrypty 09–11).

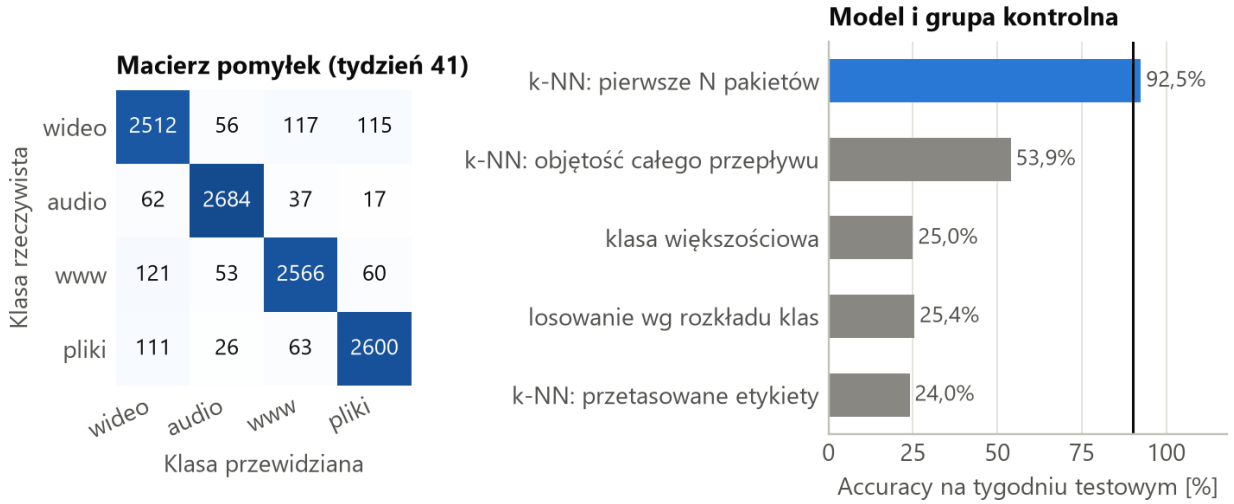
Kategorie samego zbioru są szersze od naszych, np. „Streaming media” obejmuje też sieci dostarczania czcionek. Wybraliśmy więc pojedyncze usługi odpowiadające czterem kategoriom (tabela 24). Pominęliśmy usługę apple-itunes, bo obejmuje też sklep z aplikacjami. Z każdego dnia losujemy po 400 przepływów każdej kategorii, mających co najmniej 10 pakietów z danymi. Daje to 22 400 przepływów. Model uczy się na pierwszym tygodniu (11 200 przepływów), a testujemy go na drugim (11 200).

Tabela 24. Usługi CESNET-TLS22 przypisane do kategorii ruchu

Kategoria	Usługi
wideo	amazon-prime, netflix, o2tv, twitch, vimeo, youtube
audio	soundcloud, spotify
www	bing, duckduckgo, google-www, seznam-search
pliki	cesnet-filesender, dropbox, google-drive, microsoft-onedrive, owncloud, ulozto, uschovna

Procedura jest taka sama jak dla UC Davis: blokowa walidacja krzyżowa, wybór według makro-F1, końcowy test i grupa kontrolna. Siatkę dostosowaliśmy do długości PPI: $N \in \{10, 20, 30\}$ i $k \in \{1, 3, 5, 9, 15\}$, co daje 300 konfiguracji. Foldy stratyfikujemy według kategorii, bo część usług ma zbyt mało przepływów.

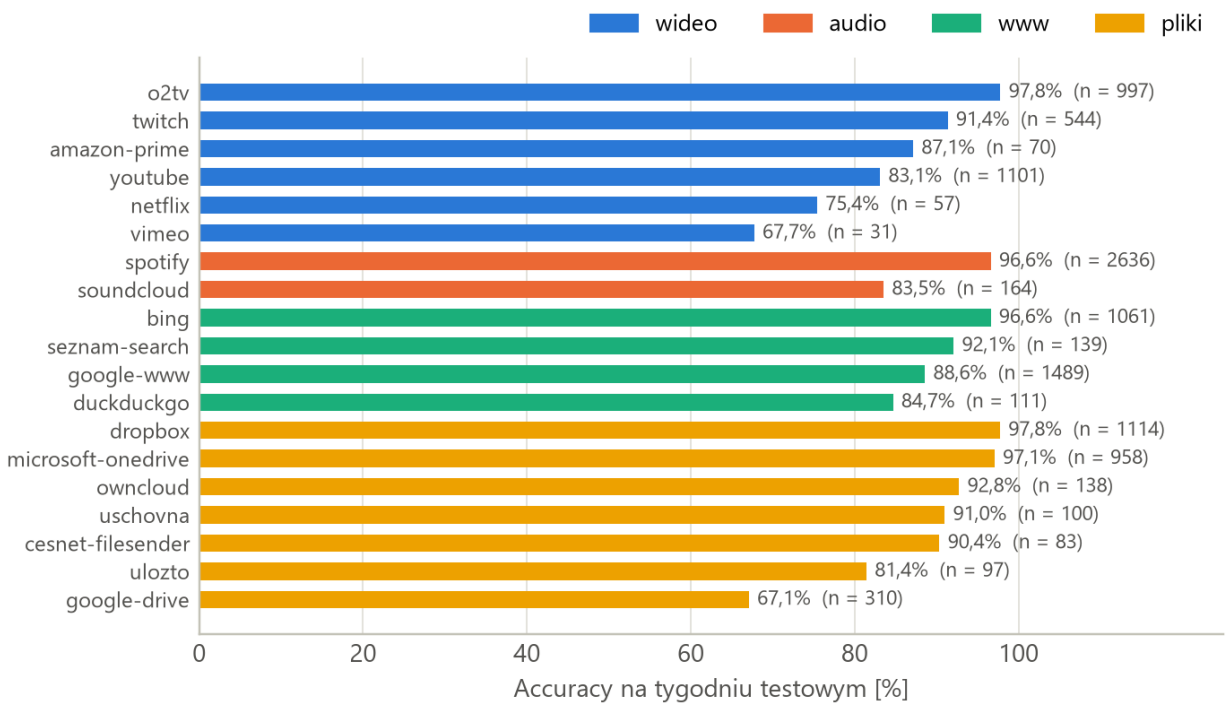
Najlepsza konfiguracja to zestaw **wszystkie** dla N = 10, k = 1, z metryką manhattańską. Na drugim tygodniu osiąga **Accuracy 92,52%** (95% CI 92,0%–93,0%) i makro-F1 92,52%, więc **kryterium 90% jest spełnione także dla TLS po TCP**. Grupa kontrolna daje 25,0% (klasa większościowa), 25,4% (losowanie) i 24,0% (przetasowane etykiety). Kontrola objętości całego przepływu osiąga tylko 53,9%, wobec 96,0% na UC Davis. Wynik na CESNET-TLS22 nie wynika więc z artefaktu objętości (rysunek 13).



Rysunek 13. Walidacja na CESNET-TLS22: macierz pomyłek i porównanie z grupą kontrolną

Najlepsze wyniki daje dziesięć pierwszych pakietów dla wszystkich zestawów cech: wszystkie 92,5%, rozmiary + odstępy 91,8%, statystyki 91,2%, rozmiary 90,9%, odstępy czasu 83,6%. Pierwsze pakiety obejmują uzgadnianie TLS, którego rozmiary zależą od serwera, np. od długości łańcucha certyfikatów. Model prawdopodobnie rozpoznaje więc w dużej mierze usługę, a przez nią kategorię ruchu. Jest to zgodne z hipotezą z podrozdziału 7.4, choć jej nie dowodzi.

Trafność dla poszczególnych usług przedstawia rysunek 14. Najsłabiej rozpoznawane są google-drive (67,1%), vimeo (67,7%) i netflix (75,4%). Etykiety pochodzą z nazwy serwera, więc przepływ oznaczony jako youtube może przenosić miniatury lub zapytania API zamiast wideo. Kategorie są przez to mniej jednorodne niż w UC Davis, co obniża trafność.



Rysunek 14. Accuracy dla usług CESNET-TLS22 na drugim tygodniu

9.4. Zagrożenia dla trafności badań

Wyniki należy interpretować z uwzględnieniem następujących zagrożeń:

- **Trafność wewnętrzna.** Każda klasa UC Davis pochodzi z innego okresu pomiaru, więc model może częściowo rozpoznawać okres, a nie aplikację. Skrypty zostawiają artefakty: stały czas trwania przepływów i dwa tryby pracy Google Drive (podrozdział 9.2). Ograniczamy je, używając tylko początku przepływu i podziału chronologicznego, ale nie usuwamy.
- **Trafność statystyczna.** Partycja ludzka liczy 83 przepływy, więc jej przedział ufności ma szerokość około 17 punktów procentowych. Wybór najlepszej z 800 konfiguracji może zawyżać wynik walidacji, dlatego raportujemy wynik na oddzielnym zbiorze testowym.
- **Trafność konstrukcyjna.** Kategorie ruchu przypisaliśmy usługom (np. Google Doc jako www), a w CESNET-TLS22 etykiety pochodzą z nazw serwerów, więc przepływ usługi wideo nie zawsze przenosi wideo.
- **Trafność zewnętrzna.** UC Davis obejmuje tylko pięć usług Google i wczesną odmianę QUIC z 2018 r., a CESNET-TLS22 jedną sieć szkieletową w 2021 r. Rozważamy zamknięty zbiór kategorii i pojedyncze przepływy, bez ruchu nieznanych usług i bez łączenia połączeń jednego użytkownika.
- **Ograniczenia symulacji obron.** Padding zmienia tylko rozmiary pakietów. Nie modelujemy wpływu obrony na czasy, retransmisje ani narzut protokołu.

9.5. Zalecenia obronne

Wyniki wskazują, że sam padding rozmiarów nie wystarcza. Nawet dopełnianie każdego pakietu do stałego rozmiaru, kosztem 36,6% dodatkowych bajtów, chroni tylko przed przeciwnikiem, który o obronie nie wie. Skuteczna obrona musi ukrywać także czasy, kierunki i objętość. Na podstawie wyników i literatury [5] proponujemy:

- **Kształtowanie ruchu w czasie**, np. wysyłanie pakietów w stałym rytmie z wypełnianiem przerw (obrony typu BuFLO [5]). Ukrywa odstępy czasu, na których najbardziej polega k-NN, ale jest najdroższe w przepustowości i opóźnieniu.

- **Wypełnianie uzgadniania połączenia**, w tym certyfikatów, ponieważ na CESNET-TLS22 już dziesięć pierwszych pakietów wystarcza do rozpoznania ruchu.
- **Łączenie wielu strumieni w jednym połączeniu** (multipleksacja w HTTP/2 i QUIC), które może utrudnić przypisanie przepływu do jednej usługi. Tego wariantu nie badaliśmy.
- **Dla administratorów sieci**: klasyfikator należy uczyć na ruchu z własnej sieci i regularnie go aktualizować, ponieważ przesunięcie danych silnie obniża trafność.

10. Podsumowanie i wnioski

Projekt sprawdził, czy bierny podsłuchujący rozpozna rodzaj ruchu w szyfrowanym kanale wyłącznie na podstawie metadanych pakietów. Odpowiedzi na pytania badawcze z podrozdziału 1.2:

- **PB1.** Tak. k-NN obserwujący 150 pierwszych pakietów osiąga na chronologicznym zbiorze testowym UC Davis Accuracy 99,46% (95% CI 98,9%–99,7%), a na ruchu TLS po TCP z CESNET-TLS22, przy 10 pierwszych pakietach, 92,52%. Kryterium Accuracy > 90% jest spełnione w obu przypadkach.
- **PB2.** Na UC Davis najlepsze są statystyki prefiksu, a surowe sekwencje wypadają słabiej. Najwięcej informacji niosą odstępy czasu: ich przetasowanie obniża makro-F1 o 0,180. Wystarcza 50 pakietów (makro-F1 w walidacji 99,05%). Na CESNET-TLS22 wystarcza 10 pakietów, czyli w praktyce uzgadnianie TLS.
- **PB3.** Artefakty zbierania danych są duże. Sama objętość całego przepływu daje 96,0% na UC Davis, ale tylko 53,9% na CESNET-TLS22. Na ruchu ludzi trafność spada do 80,7%, a klasyfikator domeny odróżnia ruch ludzi od skryptowego w 76,9% przypadków. Skrypt Google Drive pracował w dwóch trybach, a model rozpoznaje tylko zachowania obecne w danych uczących.
- **PB4.** Padding obniża trafność przeciwnika naiwnego, przy stałym rozmiarze do 37,4% kosztem 36,6% dodatkowych bajtów. Przeciwnik adaptacyjny przy każdej obronie osiąga co najmniej 96,90%, więc sam padding rozmiarów nie chroni przed atakiem.

Las losowy i SVM dają podobne wyniki jak k-NN (96,90% i 99,53% na teście głównym) i podobnie słabną na ruchu ludzi. Ograniczeniem badania nie jest więc wybór klasyfikatora, lecz reprezentatywność danych.

Szyfrowanie chroni treść, ale nie charakter komunikacji. Obrona wymaga ukrycia czasów, kierunków i objętości, a nie tylko rozmiarów pakietów. Dalsze prace powinny objąć: ruch wielu prawdziwych użytkowników, scenariusz otwartego świata z nieznanymi usługami, obrony kształtujące ruch w czasie zaimplementowane w rzeczywistych stosach TLS i QUIC oraz dłuższe sekwencje pakietów w danych CESNET.

Bibliografia

[1] J. Hayes, G. Danezis, „k-fingerprinting: A Robust Scalable Website Fingerprinting Technique”, w: *Proc. 25th USENIX Security Symposium*, 2016, s. 1187–1203.

[2] S. Rezaei, X. Liu, „How to Achieve High Classification Accuracy with Just a Few Labels: A Semi-supervised Approach Using Sampled Packets”, arXiv:1812.09761, 2018.

[3] A. Finamore i in., „Replication: Contrastive Learning and Data Augmentation in Traffic Classification Using a Flowpic Input Representation”, w: *Proc. ACM Internet Measurement Conference*, 2023, doi: 10.1145/3618257.3624820.

[4] T. Wang, X. Cai, R. Nithyanand, R. Johnson, I. Goldberg, „Effective Attacks and Provable Defenses for Website Fingerprinting”, w: *Proc. 23rd USENIX Security Symposium*, 2014, s. 143–157.

[5] K. P. Dyer, S. E. Coull, T. Ristenpart, T. Shrimpton, „Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail”, w: *Proc. IEEE Symposium on Security and Privacy*, 2012, s. 332–346, doi: 10.1109/SP.2012.28.

[6] T. Cover, P. Hart, „Nearest neighbor pattern classification”, *IEEE Transactions on Information Theory*, t. 13, nr 1, s. 21–27, 1967.

[7] E. Rescorla, „The Transport Layer Security (TLS) Protocol Version 1.3”, RFC 8446, IETF, 2018.

[8] J. Iyengar, M. Thomson, „QUIC: A UDP-Based Multiplexed and Secure Transport”, RFC 9000, IETF, 2021.

[9] M. Thomson, S. Turner, „Using TLS to Secure QUIC”, RFC 9001, IETF, 2021.

[10] E. Rescorla, K. Oku, N. Sullivan, C. A. Wood, „TLS Encrypted Client Hello”, RFC 9849, IETF, 2026.

[11] J. Luxemburk, T. Čejka, „Fine-grained TLS services classification with reject option”, *Computer Networks*, t. 220, art. 109467, 2023, doi: 10.1016/j.comnet.2022.109467.

[12] E. Biersack, C. Callegari, M. Matijasevic (red.), *Data Traffic Monitoring and Analysis*, LNCS 7754, Springer, 2013, doi: 10.1007/978-3-642-36784-7.

[13] F. Pedregosa i in., „Scikit-learn: Machine Learning in Python”, *Journal of Machine Learning Research*, t. 12, s. 2825–2830, 2011.

[14] Scapy Documentation, https://scapy.net.

[15] F. J. Massey Jr., „The Kolmogorov-Smirnov Test for Goodness of Fit”, *Journal of the American Statistical Association*, t. 46, nr 253, s. 68–78, 1951.

[16] D. Lopez-Paz, M. Oquab, „Revisiting Classifier Two-Sample Tests”, w: *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[17] L. Breiman, „Random Forests”, *Machine Learning*, t. 45, nr 1, s. 5–32, 2001.

[18] C. Cortes, V. Vapnik, „Support-vector networks”, *Machine Learning*, t. 20, nr 3, s. 273–297, 1995.

[19] J. Luxemburk, T. Čejka, „CESNET-TLS22: a large dataset for fine-grained classification of TLS services”, Zenodo, wersja 3, 2024, https://zenodo.org/records/10610895.

Uwaga: tytuł pozycji [1] podano za wydawcą, ponieważ instrukcja projektu podaje go w innym brzmieniu. Książki Biersacka i in. w brzmieniu z instrukcji nie udało się potwierdzić, dlatego cytujemy pozycję [12] tych samych autorów.

Załącznik A. Instrukcja uruchomienia

Polecenia dla Windows PowerShell, uruchamiane w katalogu repozytorium (wymagany Python 3.11 lub nowszy):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -e .
pytest
python scripts/01_download_data.py
python scripts/02_build_dataset.py
python scripts/03_explore_data.py
python scripts/04_run_knn.py
python scripts/05_plot_results.py
python scripts/06_run_padding.py
python scripts/07_analyze_shift.py
python scripts/08_run_baselines.py
python scripts/09_download_cesnet.py
python scripts/10_build_cesnet.py
python scripts/11_run_cesnet.py
python scripts/12_plot_final.py
```

Skrypt `04_run_knn.py` trwa około 5 minut, a `06_run_padding.py` około 30 minut na 16-wątkowym procesorze. Skrypty 09–11 pobierają i przetwarzają archiwum CESNET-TLS22 o rozmiarze 9,5 GB, więc są potrzebne tylko do walidacji na ruchu TLS po TCP.

Załącznik B. Dziennik zmian

Tabela 25. Dziennik zmian w dokumentacji

Data	Zakres zmian
08.11.2026	Struktura sprawozdania i spis treści
08.11.2026	Role zespołu, podział zadań, zasady pracy w repozytorium
08.11.2026	Wprowadzenie, podstawy teoretyczne, bibliografia
08.11.2026	Formalny model zagrożeń
08.11.2026	Źródła danych, kontrola jakości, analiza eksploracyjna
08.11.2026	Metodyka badawcza
08.11.2026	Wyniki k-NN, grupa kontrolna, status prac, wnioski wstępne, instrukcja uruchomienia
11.11.2026	Podział zadań na etap końcowy, numer albumu J. Wojnickiego
11.11.2026	Analiza przesunięcia danych i artefaktów zbierania ruchu
11.11.2026	Porównanie z modelami referencyjnymi i istotność cech
11.11.2026	Opis symulowanych mechanizmów paddingu
11.11.2026	Wyniki testów odporności na padding
11.11.2026	Walidacja na ruchu TLS po TCP (CESNET-TLS22)
11.11.2026	Analiza bezpieczeństwa, zagrożenia dla trafności, zalecenia
11.11.2026	Streszczenie, podsumowanie i wnioski końcowe
11.11.2026	Macierz wkładu, status realizacji, instrukcja uruchomienia